03 — Kommentare

Code für Menschen lesbar machen

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Wozu Kommentare? – Code wird gelesen, nicht nur geschrieben
2. Einzeilige Kommentare mit #
3. Unter der Haube: was macht der Interpreter mit #?
4. Docstrings mit `"""..."""`
5. Unter der Haube: Docstring vs. Kommentar
6. Guter Stil und Anti-Patterns
7. **Klassiker: Das dokumentierte Programm** – live programmiert

Lernziel: Code so dokumentieren, dass Fremde (und Sie selbst in 6 Monaten) ihn sofort verstehen – und **wissen**, was Python mit Kommentaren macht: nämlich nichts.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 03).

Wozu Kommentare?

Ohne Kommentar

```
print("Eco-Sneaker")
print("89.95 EUR")
print("auf Lager")
```

Mit Kommentar

```
# Veggie Soles -- Tagesreport (Marketing-Vorgabe)
print("Eco-Sneaker")   # unser Klassiker
print("89.95 EUR")     # Bruttopreis
print("auf Lager")
```

Code wird ca. 10-mal öfter gelesen als geschrieben. Sie schreiben heute für die Leserin von übermorgen – meist sind das Sie selbst.

Einzeilige Kommentare mit

```
# Eco-Sneaker: unser Klassiker im Sortiment
print("Eco-Sneaker")

print("Hemp-High")      # knoechelhoch, aus Hanf
```

- Ein Kommentar beginnt mit # und geht **bis zum Zeilenende**
- Zwei Plätze: **eigene Zeile** (vor dem Code) oder **am Zeilenende** (direkt neben dem Code)
- Python ignoriert alles ab # – der Text ist nur für Menschen
- **Konvention**: ein Leerzeichen nach #, zwei vor einem End-Kommentar

Unter der Haube: was macht der Interpreter mit #?

Python liest die Datei zuerst Zeichen für Zeichen ein (**Tokenisierung**):

Quelltext-Zeile	Davon wird ausgeführt
`# Tagesreport Veggie Soles`	nichts – Zeile ist für Python leer
`print("Hemp-High")  # Hanf`	`print("Hemp-High")`
`print("# kein Kommentar")`	die ganze Zeile!

- Alles ab # wird **vor** der Ausführung verworfen – die Ausführung **sieht Kommentare nie**
- Mit oder ohne Kommentare: exakt **dieselben** Schritte
- Ein # **innerhalb** von `"..."` ist Text, kein Kommentar

Kommentare kosten **null** Laufzeit und Speicher – kommentieren ist gratis.

als Werkzeug: deaktivieren und markieren

```
print("Eco-Sneaker")
# print("Bambus-Boot")   # Winter -- ausgeblendet
print("Hemp-High")

# TODO: Bambus-Boot ab Oktober wieder aufnehmen
```

- **Auskommentieren**: # davor → Zeile wird übersprungen, bleibt erhalten (IDE: STRG+/)
- **Marker**: `# TODO:` (Aufgabe), `# FIXME:` (bekannter Fehler)

▶ **Live-Demo** – `01_einzeilige_kommentare.py`

✎ **Sofort ausprobieren** – Notebook 03, Kap. 1: Lieblingsgericht-Kommentar, mittlere Zeile auskommentieren, Kommentar am Zeilenende

Docstrings mit """..."""

```
"""Veggie Soles -- Tagesreport.

Listet die Top-Produkte des Sortiments.
Autor: Anna Mueller, Stand: WiSe 2026/27
"""
print("Top 1: Eco-Sneaker")
```

- Drei Anführungszeichen `"""` öffnen und schließen den Block – beliebig viele Zeilen, kein # pro Zeile nötig
- Als **erste Anweisung einer Datei** heißt der Block **Docstring**: die offizielle Beschreibung des Programms
- Üblicher Inhalt: *Was tut das Programm? Wer hat es geschrieben?*
- `'''` funktioniert genauso – die Konvention für Doku ist `"""`

Unter der Haube: Docstring vs. Kommentar

	`# Kommentar`	`"""Docstring"""`
Beim Einlesen	wird verworfen	bleibt erhalten
Zur Laufzeit	existiert nicht	echtes **String-Objekt**
Abrufbar über	nur den Quelltext	`help(...)`, Editor-Tooltip
Typischer Ort	überall	Datei-Anfang

- Ein Docstring wird nirgends verwendet – Python **bewahrt ihn trotzdem auf**, als Beschreibung der Datei
- **Deshalb** zeigen `help(...)` und Editor ihn an – der #-Kommentar ist da längst weg

Kommentar = Notiz **im** Quelltext. Docstring = Doku, die das Programm **mit sich trägt**.

Docstrings – Demo & Übung

▶ **Live-Demo** – `02_mehrzeilige_kommentare.py` (inkl. `help(...)` auf die eigene Datei)

✎ **Sofort ausprobieren** – Notebook 03, Kap. 2: Hobbys dokumentieren, Programm-Header mit Autor und Datum schreiben

Guter Stil: WAS sagt der Code, WARUM der Kommentar

Echo-Kommentar – überflüssig

```
# gibt den Versandpreis aus
print("Versand: 4.95 EUR")
```

WARUM-Kommentar – wertvoll

```
# Pauschale: Warenwert unter 100 EUR
print("Versand: 4.95 EUR")
```

- **WAS** passiert, steht schon im Code – die Leserin kann Python lesen
- Das **WARUM** steht nirgends im Code: Geschäftsregel, Vorgabe, Absicht
- Kommentare sichern das Wissen, das sonst nur in Ihrem Kopf ist

Anti-Patterns: schlechte Kommentare

```python
print("Hallo")   # gibt Hallo aus

# print("Alter Slogan von 2024")
# print("Noch aelterer Slogan")
```

- **Echo**: wiederholt nur den Code – Rauschen, weglassen
- **Code-Leiche**: auskommentierter Alt-Code, der nie gelöscht wurde – weg damit (die Versionsverwaltung vergisst nichts)
- **Lügner**: Kommentar passt nicht mehr zum geänderten Code – schlimmer als **kein** Kommentar, er führt Leser aktiv in die Irre

▶ **Live-Demo** – `03_kommentar_stilarten.py`

Wann welche Kommentar-Art?

Situation	Werkzeug
Datei-Kopf: Zweck, Autor	Docstring `"""..."""`
WARUM an einer Zeile	`code  # Grund`
Abschnitt gliedern	`# --- Abschnitt ---`
Zeile temporär deaktivieren	# davor (STRG+/)
Aufgabe / Fehler markieren	`# TODO:` / `# FIXME:`

Faustregel: erst fragen *“steht das nicht schon im Code?”* – wenn ja, Kommentar weglassen. Wenn nein: aufschreiben, es ist gratis.

Klassiker: Das dokumentierte Programm – die Aufgabe

Der Begrüßungsdialog aus Foliensatz 02 – er läuft, aber wer versteht ihn in 6 Monaten?

```
print("==============================")
print("  Willkommen bei Veggie Soles")
print("==============================")
print("Willkommen,", input("Ihr Name: "))
print("Bis bald im Shop!")
```

Auftrag: machen Sie das Programm für Fremde lesbar –

- **Docstring-Kopf**: Was tut das Programm? Wer hat es geschrieben?
- **Abschnitts-Kommentare**: Banner / Dialog / Verabschiedung
- mindestens **ein WARUM-Kommentar**
- **Anti-Pattern-Check**: nichts Offensichtliches kommentieren

Klassiker: Das dokumentierte Programm – so läuft es

```
==============================
  Willkommen bei Veggie Soles
==============================
Ihr Name: Anna
Willkommen, Anna
Bis bald im Shop!
```

Die Ausgabe ist **exakt dieselbe** wie vorher – Kommentare ändern das Verhalten nie. Der Gewinn ist für den Rechner unsichtbar und für Menschen riesig.

Klassiker: Das dokumentierte Programm – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_dokumentiertes_programm.py`
Teilziele:

1. Programm aus Foliensatz 02 abtippen – läuft es?
2. Docstring-Kopf: Zweck + Autor
3. Abschnitts-Kommentare: Banner / Dialog / Verabschiedung
4. Ein WARUM-Kommentar: warum steckt `input()` im `print()`?
5. Anti-Pattern-Check: alles Offensichtliche wieder raus

✎ **Zum Selberlösen** – Handout *“Klassiker 03: Das dokumentierte Programm”* (`Klassiker/03_Klassiker_Dokumentiertes_Programm.pdf`). Vertiefung: Notebook 03, Trainingsmaterial Aufgabe 4–6.

Cheat Card

Aufgabe	Syntax
Kommentar (eigene Zeile)	`# Erklaerung`
Kommentar am Zeilenende	`code  # Erklaerung`
Zeile deaktivieren	`# print(...)`
Docstring (Datei-Kopf)	`"""Zweck, Autor..."""`
Aufgabe / Fehler markieren	`# TODO:` / `# FIXME:`
Klassiker heute	Docstring + Abschnitte + 1× WARUM

Merksatz: *WAS* sagt der Code, *WARUM* sagt der Kommentar – und was der Interpreter nie sieht, kostet auch nichts.

Ausblick: Notebook 04 – Variablen

Unser dokumentiertes Programm hat eine Schwäche: es kann sich den Namen **nicht merken** – nach der `print`-Zeile ist die Antwort weg. Für jede weitere Verwendung müssten wir erneut fragen.

Nächste Woche bekommt jeder Wert einen **Namen** (Sneak-Peek – Syntax lernen wir dann):

```
name = input("Ihr Name: ")   # Wert bekommt Namen
print("Willkommen,", name)   # ... beliebig oft nutzbar
```

Damit werden auch Kommentare mächtiger: gute **Variablennamen** ersetzen viele Kommentare.

Heute geübt

- ✓ Wozu Kommentare: Code wird 10-mal öfter gelesen als geschrieben
- ✓ Einzeilige Kommentare mit # – eigene Zeile und Zeilenende
- ✓ Unter der Haube: # wird bei der Tokenisierung verworfen, kostet nichts
- ✓ Docstrings `"""..."""` – Doku, die das Programm mit sich trägt
- ✓ Guter Stil: WARUM statt WAS, Anti-Patterns erkannt
- ✓ **Klassiker: Das dokumentierte Programm** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Das dokumentierte Programm** eigenständig lösen (Handout)
- “Sofort ausprobieren”-Aufgaben im Notebook 03 (Kap. 1 + 2)
- Trainingsmaterial: gestufte Aufgaben mit Musterlösungen